kiln: a container runtime for short-lived, untrusted AI agent workloads

White paper

Stephen Estrada

October 2026

Abstract

AI agents run code as a long series of short tool calls: a shell command, a test run, a script the model just wrote. Each call is untrusted, each one blocks the agent, and between calls the sandbox sits idle. kiln is a Linux container runtime built for exactly that pattern. On the same VM and root filesystem as crun, runc and Docker, it starts a sandbox in 13.2 ms cold (p50), the same as crun and 6.4× faster than Docker; with networking, 22.2 ms against Docker's 141.8 ms. It runs a command in an existing sandbox in 3.5 ms, against 9.7 ms for crun, 24.1 ms for runc and 53.7 ms for Docker. An idle sandbox costs the host about 0.4 MiB and needs no daemon. It does this with a user namespace, a seccomp filter, a per-sandbox disk quota and an egress policy turned on by default. Once code is running, every runtime is equally fast, because it is the same kernel; kiln's advantage is entirely in the cost around the code, and this paper claims nothing beyond what was measured.

Table 1: Summary. Times are p50 of 100 runs (ms); memory is the growth of host memory per extra idle sandbox (KiB, two runs of 50). Details and conditions in §4 and §5.

	kiln	crun	runc	Docker
Cold start, no network	**13.2**	13.4	34.2	84.0
Cold start, with network	**22.2**	—	—	141.8
Warm start (exec `true`)	**3.5**	9.7	24.1	53.7
Memory per idle sandbox (KiB)	370–379	239–411	598–760	5328–5600
Daemons that must run	none	none	none	≈160.1 MiB
User namespace, seccomp, disk quota	all three	none	none	seccomp
Steady-state CPU and I/O		the same within noise		

1 The problem: agents pay the overhead on every step

An agent works in a loop: the model decides on an action, a tool runs it, the result goes back to the model, and the model decides again. Tools that touch the system (run a command, install a package, execute generated code) need a sandbox, because the code was written by a model or downloaded from somewhere nobody checked. Three properties of this workload decide what a sandbox runtime should optimize:

1. **Latency multiplies by the number of steps.** The loop blocks on every tool call, so whatever the runtime adds is paid per step, not per session. For illustration only: a session of 50 tool calls run with `docker exec` spends $50 \times 53.7\,\text{ms} \approx 2.7\,\text{s}$ in the runtime alone; with `kiln exec`, $50 \times 3.5\,\text{ms} \approx 0.18\,\text{s}$.
2. **Sandboxes are mostly idle.** Between calls the model is thinking, often for seconds. What a sandbox costs while idle, times the number of concurrent sessions, is what a host pays.
3. **The code is untrusted.** Isolation has to be the default, not a flag someone remembers, and it should not have to be traded for latency.

What does *not* differ between runtimes is how fast the code runs once started: a container is a set of kernel features around ordinary processes, so a CPU-bound or I/O-bound workload runs on the same kernel whichever runtime set it up (§4.4). A runtime can only win on the cost around the code: starting, idling, running the next command and tearing down.

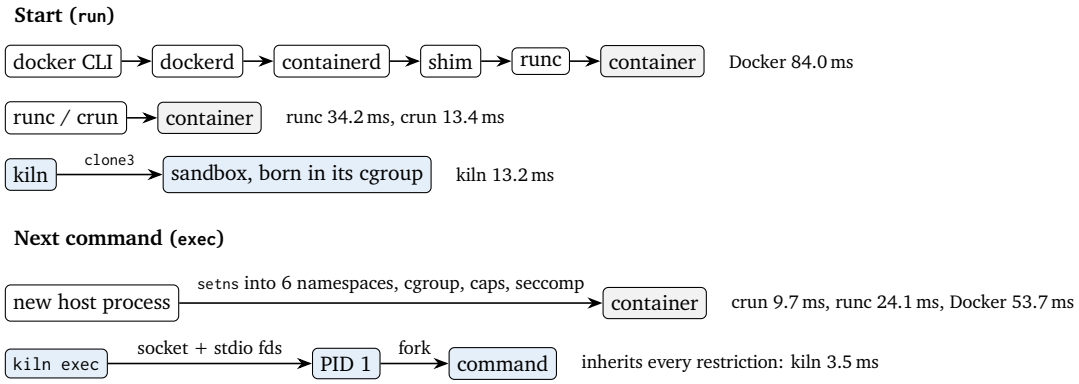


Figure 1: The path from a request to running code. Docker crosses an API and two daemons; runc and crun are single processes. For the next command in an existing sandbox, the others start a host process that joins the container from outside; kiln asks the sandbox's own PID 1 to fork it. Times are cold p50.

2 Why general-purpose runtimes pay for things agents don't need

General-purpose runtimes are built for long-running services managed by people and orchestrators, and their costs follow from that (Figure 1).

Daemon layers. `docker run` goes from the CLI to `dockerd` over its API, to `containerd`, to a per-container shim, and finally to runc, which does the actual work. Those layers buy remote management, logging drivers, restart policies and a stable API, none of which a sandbox for one tool call uses. They show up as start time (84.0 ms for Docker against 34.2 ms for runc, which Docker uses underneath, though Docker also prepares an overlay snapshot and a seccomp profile), as resident memory (`dockerd` and `containerd` together use about 160.1 MiB PSS whether one container runs or none), and as about 11 MiB of private memory per container for its Go shim and PID 1 together (11228 KiB USS).

Generic image handling. Docker and containerd manage OCI images: content-addressed layers, snapshotters, registries. That machinery exists to distribute images between machines. An agent sandbox starts from one image that is already on the host, so kiln uses that image directly as the read-only lower layer of an overlay, with a per-sandbox writable layer on top.

Joining from outside. To run another command in an existing container, `runc exec` and `crun exec` start a new host process that enters the container's namespaces one by one with `setns`, joins its cgroup, sets capabilities and seccomp, and only then executes the command. runc, a Go program, joins the cgroup by writing to `cgroup.procs`, which migrates a running process and waits for an RCU grace period; crun creates the process inside the cgroup with `clone3`. Docker adds its daemon path on top of runc.

Grace periods on the critical path. Making the kernel's RCU grace periods expedited, a host-wide setting, cut runc's cold start from 33.6 ms to 27.3 ms and kiln's former build's from 29.0 ms to 17.0 ms: both were waiting for grace periods. Current kiln and crun are unaffected (13.2 ms vs 13.3 ms; crun 13.2 ms vs 13.4 ms).

3 Design choices that buy speed

No daemon. `kiln run` is one process that sets up the sandbox, waits for it and removes it. All state lives in the kernel and a few directories, named by a run ID. A persistent sandbox (`kiln create`) gets a small supervisor of its own, a shim, which owns its cleanup; the shim and the sandbox's PID 1 run the same binary and share almost all their pages, so the two together cost 136 KiB of private memory (USS) against Docker's 11228 KiB for its shim and PID 1.

Born in the cgroup. kiln creates the sandbox with `clone3(CLONE_INTO_CGROUP)`, so the child exists inside its cgroup from its first instruction and no running process is ever migrated. The older approach, writing the PID to `cgroup.procs`, cost 5–15 ms of grace-period waiting on every cold start and was the largest single step in kiln's own before-and-after (Figure 4).

Talking to the kernel directly. Starting a helper program costs milliseconds before it does anything: a cold `nft` takes 3.9 ms just to load its libraries. kiln configures links, addresses and routes with its own rtnetlink client

Table 2: What each runtime sets up in the benchmarks (from `docs/benchmarks.md`).

	kiln	`docker run --rm`	runc / crun (bare spec)
Path to the kernel	one process	CLI, dockerd, containerd, shim, runc	one process
Root filesystem	overlay on XFS, per-run quota	overlay snapshot	unpacked tree, read-only
Namespaces	PID, mount, UTS, IPC, net, cgroup, **user**	same, no user ns	same, no user ns
cgroup limits	memory, swap, CPU, PIDs, OOM group kill	memory, swap, CPU, PIDs	memory, swap, CPU, PIDs
Syscall filter	seccomp denylist, `no_new_privs`	default seccomp profile	none
PID 1	kiln-init (signals, reaping)	the command	the command
Network (networked cases)	veth, bridge, nft chain, DNS policy	veth on docker0, iptables NAT	loopback only

instead of `ip` and `nsenter`, runs exactly one `nft` per networked start (its policy batch ends with a listing that is checked against the shared base ruleset, and repairs are committed only on drift), sends teardown deletions as one nfnetlink batch whose socket is closed by a short-lived child so kiln does not wait for the kernel's deferred freeing, and leaves the host end of the veth pair to the dying network namespace.

Exec through PID 1. A persistent sandbox's PID 1 holds a Unix socket, opened before the sandbox's root was swapped. `kiln exec` sends the arguments and passes its stdin, stdout and stderr as file descriptors; PID 1 forks the command onto them and returns its exit code. The command is born with every namespace, the cgroup, the seccomp filter and the Landlock policy, because it inherits them. Nothing joins from outside, so there is nothing to re-apply.

Security on by default, inside the measured time. Every number in this paper includes what kiln sets up by default and the others, as benchmarked, did not (Table 2): a user namespace that makes sandbox root an unprivileged host UID, a seccomp filter with `no_new_privs`, a per-sandbox XFS disk quota, and an init process that forwards signals and reaps orphans. Networked sandboxes get an nftables egress policy (no private ranges with `--net full`; only listed destinations with `--allow`) and a DNS forwarder that resolves only allowed names. A Landlock read-only mode is available as an option; it was not part of the benchmark runs.

4 Results

Method. Each number is the wall time of one command measured with hyperfine without a shell: 10 warm-up runs, then 100 timed runs, with 100 ms of idle before each ("cold"; back-to-back runs reuse warm kernel paths and hide real costs). Every runtime starts the same Alpine 3.24.2 minirootfs with kiln's default limits (512 MiB memory without swap, 1 CPU, 128 PIDs). The workload is `true`, so the time is the runtime's own. Environment: MacBook Air (Apple M2), Lima VM (vz, 4 vCPU, 4 GiB), Ubuntu 24.04, Linux 6.8.0-146 aarch64; Docker 29.1.3 with runc 1.3.4, runc 1.3.4, crun 1.14.1, kiln release build.

4.1 Cold start

Without a network, kiln and crun tie: p50 13.2 ms and 13.4 ms, p95 15.8 ms and 15.8 ms, p99 16.2 ms and 16.7 ms. runc takes 34.2 ms (2.6×) and Docker 84.0 ms (6.4×), Figure 2. With a network, kiln takes 22.2 ms (p95 28.4 ms) against Docker's 141.8 ms (p95 156.1 ms), 6.4× faster, while installing a per-sandbox firewall chain and DNS policy.

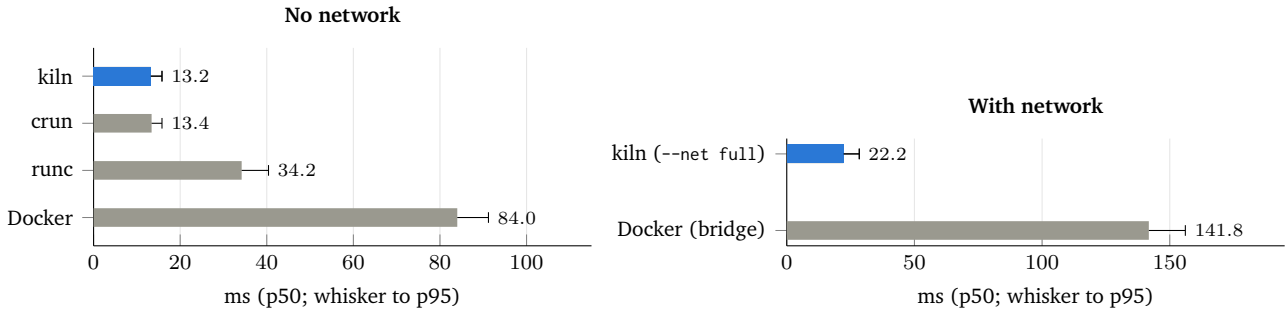


Figure 2: Cold start of `true`, 100 runs each. runc and crun were measured only without a network: their bare configuration has loopback only.

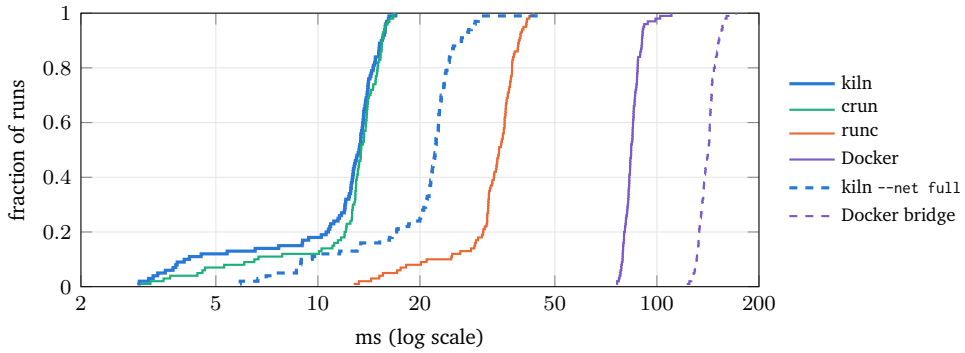


Figure 3: Distribution of the cold-start runs. A few runs of the fastest runtimes finished in a few milliseconds (13 of 100 for kiln, 8 for crun, under 6 ms): the 100 ms pause did not always let the system go fully idle. Reported figures are percentiles of all 100 runs.

Where kiln's time went. Before this work, kiln took 28.9 ms cold without a network and 87.9 ms with one; back to back, the same build took 2.7 ms, which is why measuring cold mattered. Figure 4 shows each change, all builds measured in one session. Two causes explain almost all of the old time: waiting for RCU grace periods (migrating into the cgroup, closing nftables sockets after deleting commits, deleting the veth) and starting helper processes. What remains without a network is mostly creating the namespaces (1.6 ms), the sandbox's mounts (2.5 ms), starting the binary, and about 5 ms between the command exiting and the host seeing the sandbox gone; a network adds one `nft` process and the veth (Table 3).

Table 3: Where a cold start's time goes: `KILN_TRACE` phases, medians of 15 cold runs in ms, before and after this work (from `docs/benchmarks.md`).

	`--net none`		`--net full`	
Phase	before	now	before	now
Image check, storage, quota, overlay directories	1.2	1.5	1.5	1.3
cgroup create	0.6	0.9	0.8	0.9
Clone with five namespaces	1.3	1.6	1.6	1.6
Move the child into its cgroup	**8.6**	gone	**9.9**	gone
Network setup on the host	4.5	—	**42.3**	7.5
Child setup (mounts, `pivot_root`, `/proc`, user ns)	∼2	∼2.5	∼2	∼2.4
Command runs and exits	1.4	1.6	0.9	1.0
Sandbox exit seen by the host	5.0	5.3	5.1	5.3
Network cleanup	0.1	0.1	**21.4**	0.3
cgroup and run directory cleanup	0.3	0.3	0.2	0.3

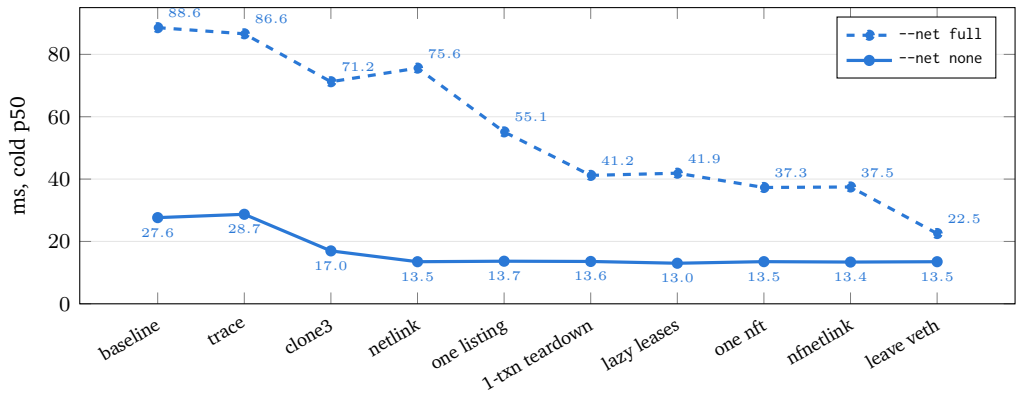


Figure 4: Cold-start p50 after each change, every build measured in the same session (`bench/results/steps`). Some changes only show in combination; `docs/benchmarks.md` explains each.

4.2 Warm start: the next command

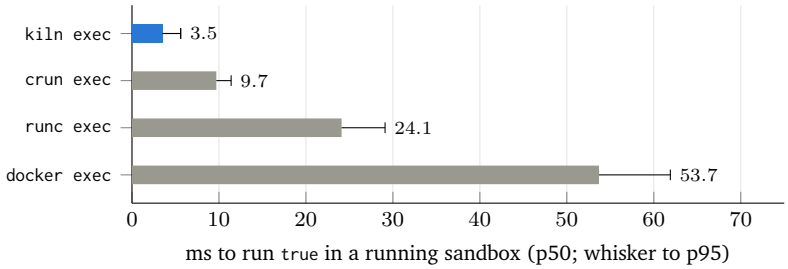


Figure 5: Warm start, 100 runs each with 100 ms idle before each run. kiln: a `kiln create` sandbox; others: a container running `sleep infinity`.

For an agent session the sandbox is created once and each tool call runs in it, so warm start is the latency the agent sees on almost every step. `kiln exec` takes 3.5 ms (p99 5.9 ms), against 9.7 ms for crun (p99 12.0 ms), 24.1 ms for runc (p99 33.6 ms) and 53.7 ms for Docker (p99 63.3 ms), Figure 5: 2.8×, 6.9× and 15.3× kiln's time. With a network the numbers barely move (kiln 3.8 ms, Docker 54.1 ms). The difference is the path in Figure 1: a fork inside the sandbox against a new process joining from outside.

4.3 Idle overhead

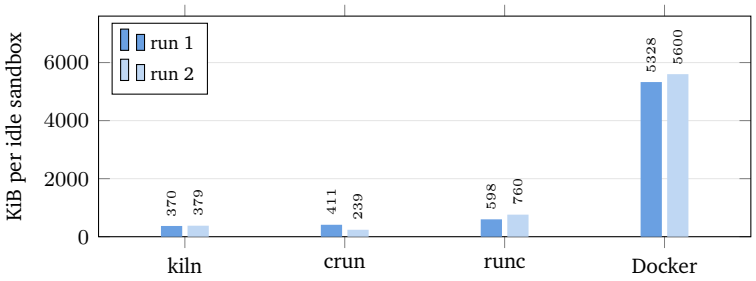


Figure 6: Growth of host memory in use (`MemTotal − MemAvailable`, page cache dropped) per sandbox when 50 idle sandboxes are started, two runs. Coarse (about ±200 KiB) but clear in order of magnitude. Not shown: Docker's daemons, about 160.1 MiB PSS, paid once.

Fifty idle sandboxes grew the host's memory use by 370 and 379 KiB per sandbox for kiln in two runs, against 411 and 239 for crun, 598 and 760 for runc, and 5328 and 5600 for Docker (Figure 6). This counts kernel objects too: namespaces, mounts, page tables and the cgroup. kiln keeps a supervisor per persistent sandbox

and still lands with crun and runc, which keep none: after start, their container is reparented to the host's init and nothing is left to clean up after it or to serve exec. Docker needs `dockerd` and `containerd` running in addition, about 160.1 MiB PSS; kiln needs no daemon to start a sandbox (its DNS forwarder, 1109 KiB PSS, is needed only for name lookups from networked sandboxes).

4.4 Steady state: the same everywhere

Two single-threaded workloads under the same 1-CPU limit, timed inside the sandbox so start-up is excluded, 7 runs each, interleaved: hashing 512 MiB of zeros, and writing 1 GiB to the sandbox's writable layer with `fsync`. CPU time is identical; I/O overlaps across all four with wide ranges, because the VM's disk is a file on the Mac's SSD (Table 4). Where writes land differs by configuration (kiln's upper layer is on XFS in a loop device for its disk quota; Docker's on an overlay snapshot; runc's and crun's straight into the unpacked tree), which is a storage choice, not a runtime one.

Table 4: Steady state: median seconds (min–max) over 7 runs.

	kiln	crun	runc	Docker
CPU-bound	2.93 (2.92–2.94)	2.93 (2.90–3.01)	2.93 (2.92–2.94)	2.94 (2.92–3.00)
I/O-bound	1.87 (1.60–2.20)	2.01 (1.74–2.90)	1.66 (1.38–1.77)	1.71 (1.34–2.04)

5 Fairness and limitations

- **One machine, one VM.** Every number comes from a Lima VM (4 vCPU, 4 GiB) on an M2 MacBook Air: aarch64, Linux 6.8, Ubuntu 24.04. The Mac was otherwise in normal use, and results from different sessions differ by a few milliseconds; tables that compare kiln builds were measured in one session. The VM's vCPUs sleep when idle, so cold runs include waking them. Nothing was measured on x86-64 or on bare metal, and the benchmarks were run by kiln's author.
- **Not identical work.** Table 2 lists what each runtime sets up. kiln does more isolation than the bare runc and crun configurations (user namespace, seccomp, overlay with quota, an init process), and Docker more management. The networked comparison is kiln's nftables policy against Docker's iptables NAT; runc and crun had no networked case. Docker was configured with `ip-forward-no-drop` so its FORWARD policy would not drop kiln's traffic.
- **Bimodal cold runs.** A few cold runs of kiln and crun completed in a few milliseconds (Figure 3); percentiles over all 100 runs are reported, and the p50 is not affected by them.
- **Coarse memory.** Host-wide memory growth is good to about ±200 KiB per sandbox; per-process RSS, PSS and USS for one sandbox are in `docs/benchmarks.md`. Steady state used 7 runs per case.
- **A trivial workload.** `true` isolates the runtime's cost. A real tool call adds its own run time, the same under every runtime.
- **Scope.** kiln has no OCI image or runtime-spec compatibility, runs on a single host, and has no orchestrator integration yet. Its isolation is a shared-kernel container's: a kernel bug reachable through an allowed syscall is still an escape.

6 Future work

- **Pre-warmed pools.** A daemon that creates sandboxes ahead of time and hands one out on request would turn a cold start into an exec: 3.5 ms instead of 13.2 ms–22.2 ms. Networked pool members could have their veth pair and firewall chain prepared ahead too. This needs the planned daemon and a policy for pool size.
- **Smaller steps already measured.** Loading libnftables in-process instead of starting `nft` measured 0.55 ms to load and 1.2 ms to apply a policy, against 4–6 ms for the process: 3–4 ms per networked start. Returning the exit code before teardown would save most of the 5 ms exit phase, at the cost of cleanup finishing after `kiln run` returns.
- **Snapshot and restore** of a prepared sandbox (installed packages, warm files) so a session can start from where a previous one left off; the overlay's upper layer already records exactly what a session changed.
- **Multiple hosts**: a daemon per node and a Kubernetes operator that places sandboxes, with the same per-node design.

- **Broader measurements**: x86-64, bare metal, more concurrency, and real agent workloads rather than `true`.

7 Conclusion

A sandbox for agents is judged on the cost around the code: how fast it starts, how fast it runs the next command, and what it costs while it waits. Measured on the same VM and root filesystem, kiln starts as fast as crun, runs the next command 2.8× faster than crun and 15.3× faster than Docker, and idles at a few hundred KiB with no daemon, while turning on a user namespace, seccomp, a disk quota and an egress policy by default. It does not make code run faster once started; no runtime can, because the kernel does that part.